TUGAS 3
BASIS DATA

Praktikum Transaksi dan Praktikum Basis Data Terdistribusi Pada MySQL

Nama : **Subhan Yoga Maulana**

NIM : **044876223**

Program Studi : **Sistem Informasi**

Link Youtube

1. Tujuan Praktikum

Praktikum Tugas 3 Mata Kuliah Basis Data (MSIM4206) ini bertujuan untuk mempraktikkan secara langsung dua materi utama dalam pengelolaan basis data relasional menggunakan MySQL/MariaDB pada XAMPP dan VirtualBox Ubuntu. Bagian pertama adalah Praktikum Transaksi yang menerapkan prinsip ACID (Atomicity, Consistency, Isolation, Durability) menggunakan Stored Procedure (Routines) dengan perintah START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT, serta pembuatan tabel relasi dan Trigger untuk otomatisasi pengurangan stok obat. Bagian kedua adalah Praktikum Basis Data Terdistribusi menggunakan mekanisme Replikasi Master-Slave satu arah dari sistem operasi Windows sebagai Server Master (IP 192.168.56.1) ke sistem operasi Linux Ubuntu pada VirtualBox sebagai Server Slave (IP 192.168.56.101).

2. Ketentuan dan Persiapan Lingkungan Praktikum

Spesifikasi perangkat lunak dan skema basis data yang digunakan adalah sebagai berikut:

- Basis Data Awal (Melanjutkan Tugas 2): rumah_sakit yang terdiri dari 6 tabel awal yaitu administrator, daftar, dokter, dokter_admin, pasien, dan pasien_dokter dengan storage engine InnoDB.
- Penambahan Tabel Tugas 3: tabel obat (id_obat, kode_obat, nama_obat, harga, stok) dan tabel transaksi_obat (id_transaksi, id_pasien, id_obat, jumlah, total_harga) yang dihubungkan melalui Foreign Key ON DELETE CASCADE dan ON UPDATE CASCADE.
- Stored Procedures (Routines): transaksi1 (tanpa COMMIT), transaksi2 (dengan COMMIT), transaksi3 (ROLLBACK pada UPDATE), transaksi4 (Modifikasi

ROLLBACK pada INSERT dan COMMIT pada UPDATE), serta transaksi5 (SAVEPOINT point1 dan ROLLBACK TO point1).

- Trigger: tg_stok_obat dengan waktu AFTER dan event INSERT pada tabel transaksi_obat untuk mengurangi stok obat secara otomatis.
- Topologi Replikasi Master-Slave: Server Master pada Host Windows (IP 192.168.56.1, server-id = 1, log-bin = mysql-bin, binlog-do-db = rumah_sakit) dan Server Slave pada Guest Ubuntu VirtualBox (IP 192.168.56.101, server-id = 2, replicate-do-db = rumah_sakit).

3. Kode SQL Praktikum Transaksi, Tabel Relasi, dan Trigger

Berikut adalah kode SQL lengkap yang digunakan pada pelaksanaan Praktikum Transaksi dan Trigger:

```
-- ============================================================
-- 1. STORED PROCEDURE transaksi1 (START TRANSACTION Tanpa COMMIT)
-- ============================================================
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (6, 'Hanif', 'Jl. Babakan Loa No. 22', 'L');
SELECT * FROM pasien;
END

-- ============================================================
-- 2. STORED PROCEDURE transaksi2 (Dengan Perintah COMMIT)
-- ============================================================
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (6, 'Hanif', 'Jl. Babakan Loa No. 22', 'L');
COMMIT;
SELECT * FROM pasien;
END

-- ============================================================
-- 3. STORED PROCEDURE transaksi3 (Perintah ROLLBACK)
-- ============================================================
BEGIN
START TRANSACTION;
UPDATE pasien SET nama_pasien = 'Septin' WHERE id_pasien = 3;
ROLLBACK;
COMMIT;
SELECT * FROM pasien;
END

-- ============================================================
-- 4. STORED PROCEDURE transaksi4 (Modifikasi ROLLBACK & COMMIT)
-- ============================================================
```

```
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (7, 'Rudi', 'Jl. Kebon Jeruk No. 14', 'L');
ROLLBACK;
UPDATE pasien SET nama_pasien = 'Septin' WHERE id_pasien = 3;
COMMIT;
SELECT * FROM pasien;
END

-- ============================================================
-- 5. STORED PROCEDURE transaksi5 (SAVEPOINT point1 & ROLLBACK TO point1)
-- ============================================================
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (8, 'Dudung', 'Jl. Cimahi No. 11', 'L');
SAVEPOINT point1;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (9, 'Maman', 'Jl. Garut No. 5', 'L');
ROLLBACK TO point1;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`,
`jenis_kelamin`)
VALUES (10, 'Ujang', 'Jl. Tasikmalaya No. 9', 'L');
COMMIT;
SELECT * FROM pasien;
END

-- ============================================================
-- 6. PEMBUATAN TABEL OBAT, TRANSAKSI_OBAT, DAN TRIGGER tg_stok_obat
-- ============================================================
CREATE TABLE obat (
  id_obat INT(6) NOT NULL AUTO_INCREMENT PRIMARY KEY,
  kode_obat VARCHAR(5) NOT NULL,
  nama_obat VARCHAR(30) NOT NULL,
  harga INT(10) NOT NULL,
  stok INT(5) NOT NULL
) ENGINE=InnoDB;

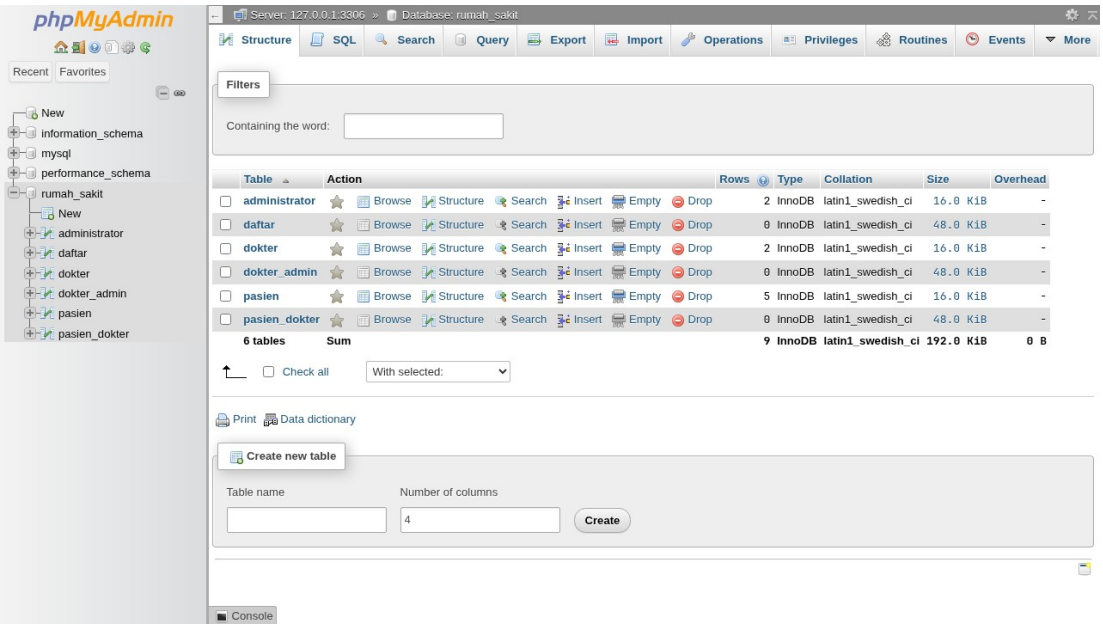
CREATE TABLE transaksi_obat (
  id_transaksi INT(6) NOT NULL AUTO_INCREMENT PRIMARY KEY,
  id_pasien INT(3) NOT NULL,
  id_obat INT(6) NOT NULL,
  jumlah INT(5) NOT NULL,
  total_harga INT(10) NOT NULL,
  INDEX (id_pasien),
  INDEX (id_obat),
  CONSTRAINT fk_to_pasien FOREIGN KEY (id_pasien) REFERENCES
pasien(id_pasien) ON DELETE CASCADE ON UPDATE CASCADE,
```

```
  CONSTRAINT fk_to_obat FOREIGN KEY (id_obat) REFERENCES obat(id_obat) ON
DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

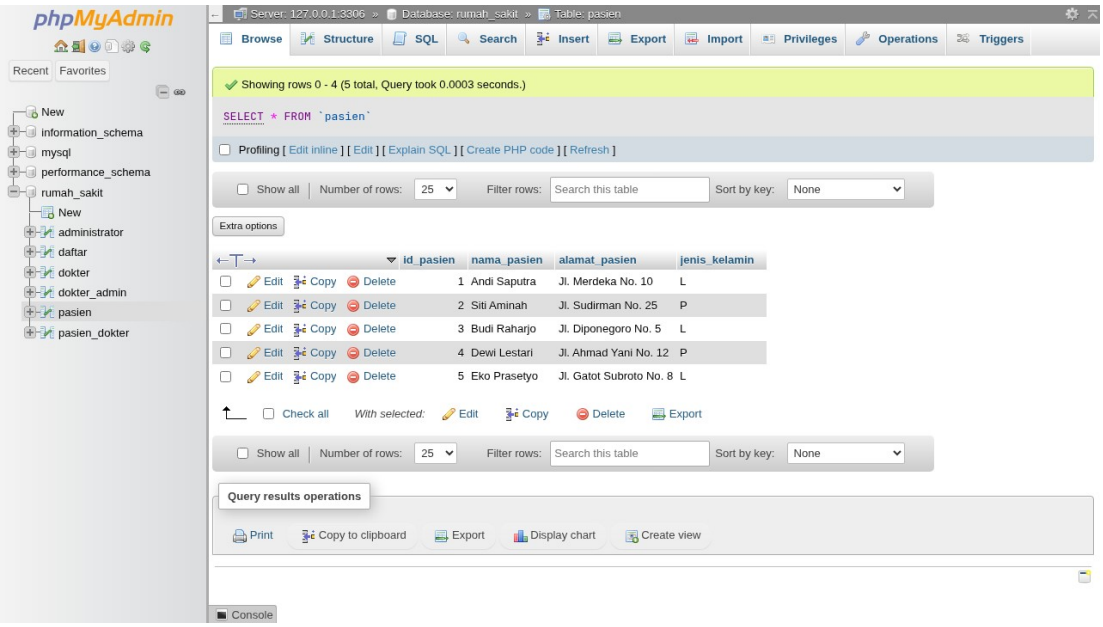
-- Definisi Trigger tg_stok_obat (AFTER INSERT ON transaksi_obat):
UPDATE obat SET stok = stok - NEW.jumlah WHERE id_obat = NEW.id_obat;
```

4. Dokumentasi Langkah demi Langkah Praktikum A (Transaksi & Trigger)

4.1. Pemeriksaan Database Awal rumah_sakit (Hasil Tugas 2): Sebelum membuat routine transaksi, dipastikan seluruh 6 tabel awal menggunakan storage engine InnoDB dan tabel pasien memiliki 5 baris data awal (id_pasien 1 sampai 5).

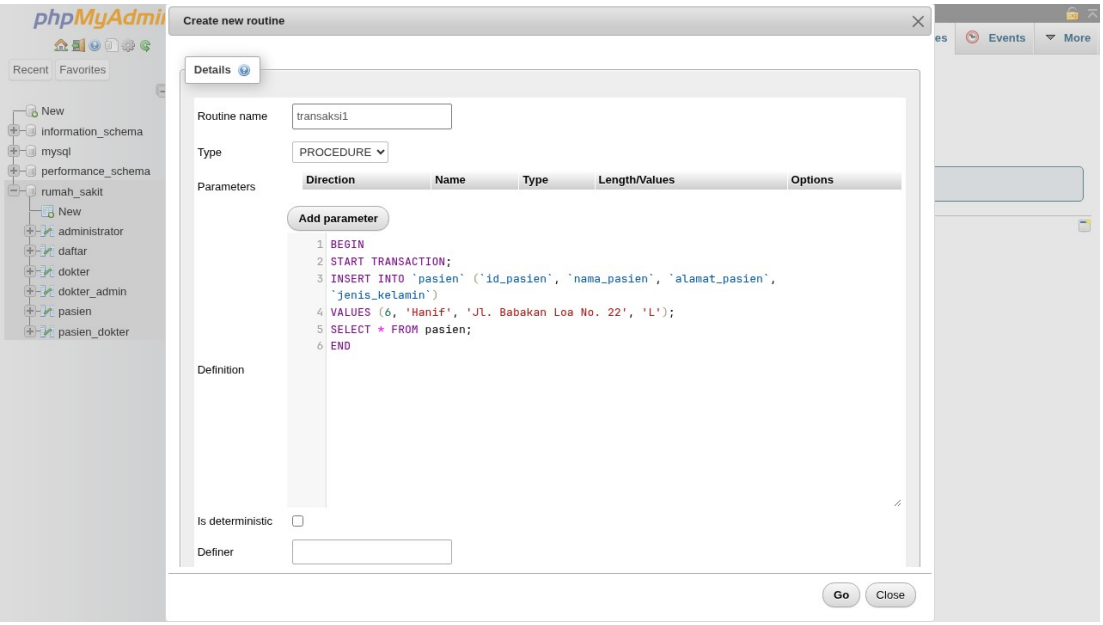


Gambar 1. Struktur 6 tabel awal pada database rumah_sakit (melanjutkan Tugas 2) dengan storage engine InnoDB.

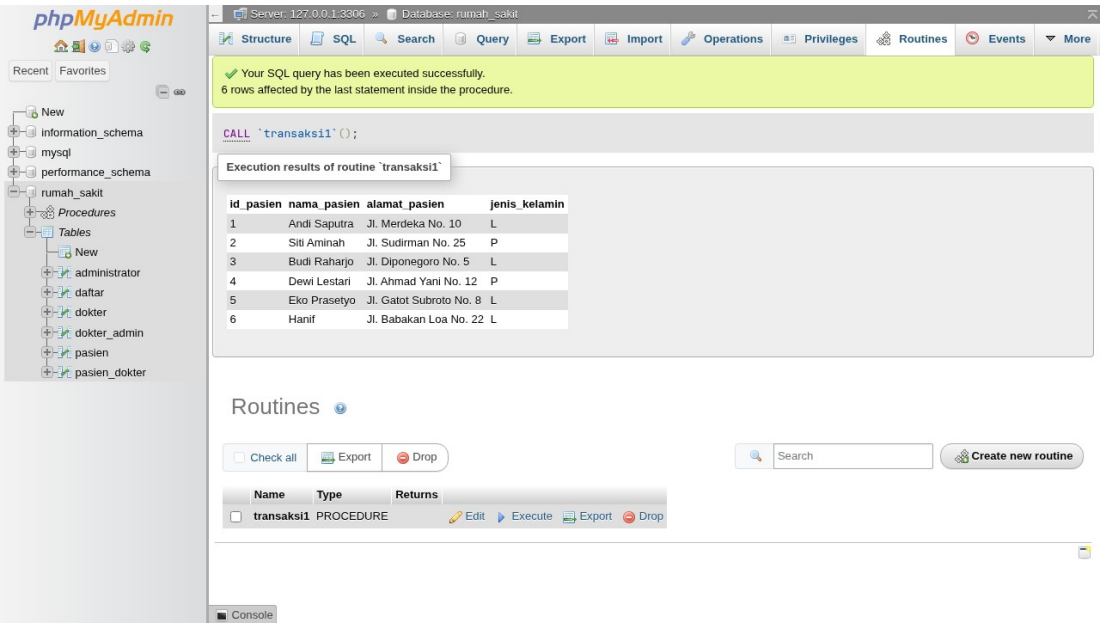


Gambar 2. Isi awal tabel pasien sebelum pembuatan Stored Procedure transaksi (5 data pasien).

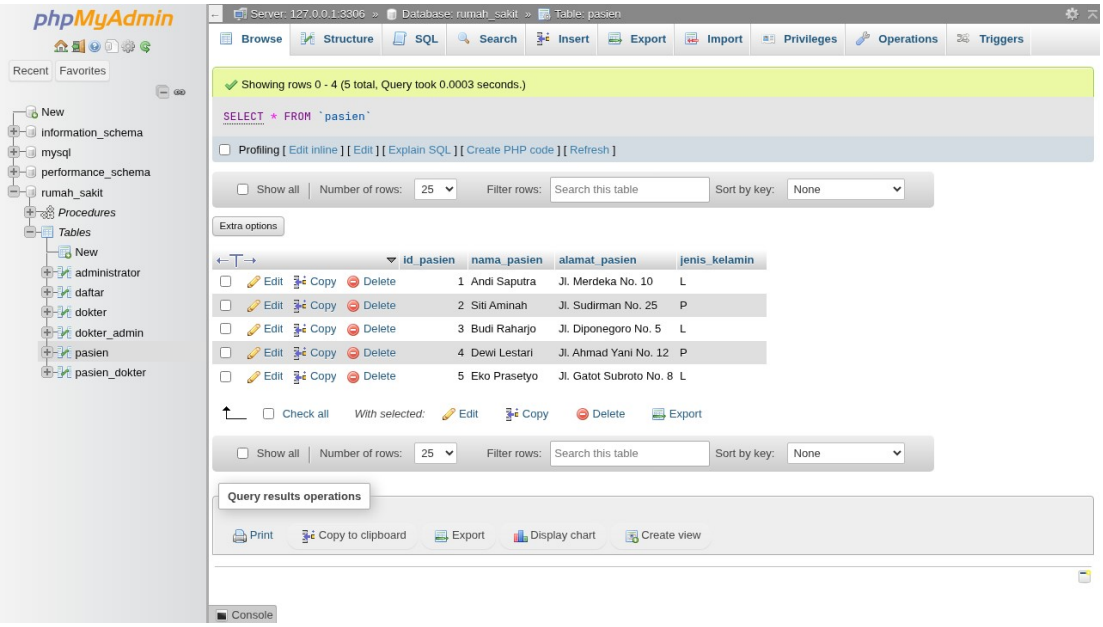
4.2. Pembuatan dan Eksekusi Routine transaksi1 (Tanpa Perintah COMMIT): Melalui menu Routines -> Create new routine, dibuat prosedur transaksi1 yang menyisipkan pasien ID 6 (Hanif) tanpa diakhiri COMMIT. Saat dieksekusi, data Hanif tampil pada hasil query prosedur, tetapi ketika tabel pasien diperiksa pada menu Browse, data ID 6 belum tersimpan permanen.



Gambar 3. Pembuatan Stored Procedure transaksi1 pada jendela Create new routine (tanpa COMMIT).

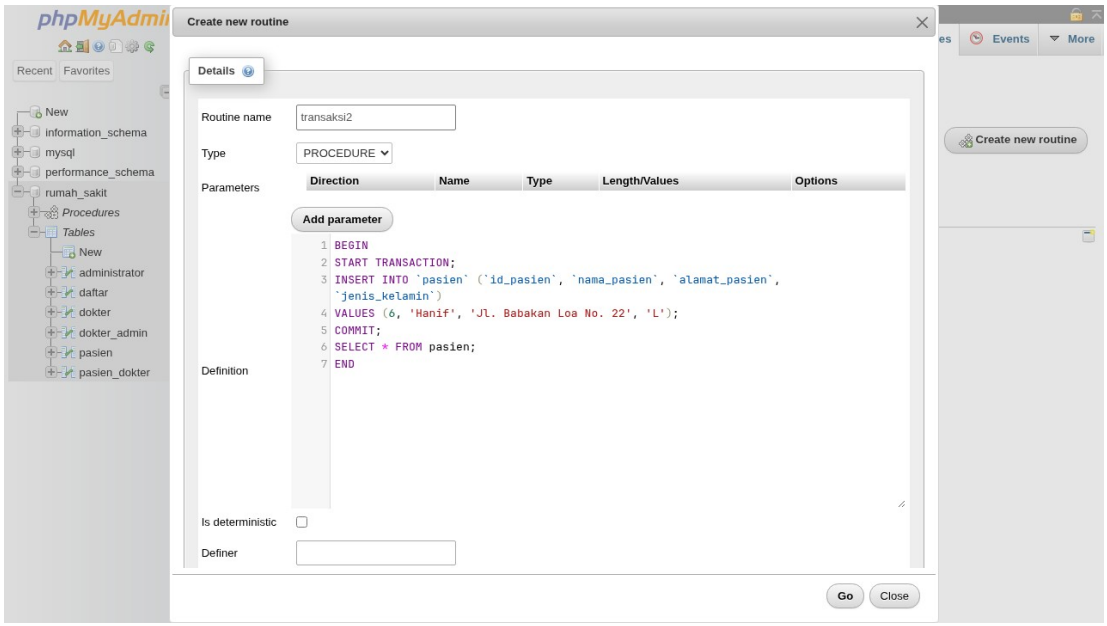


Gambar 4. Hasil klik Execute pada transaksi1 menampilkan baris ID 6 (Hanif) secara sementara.

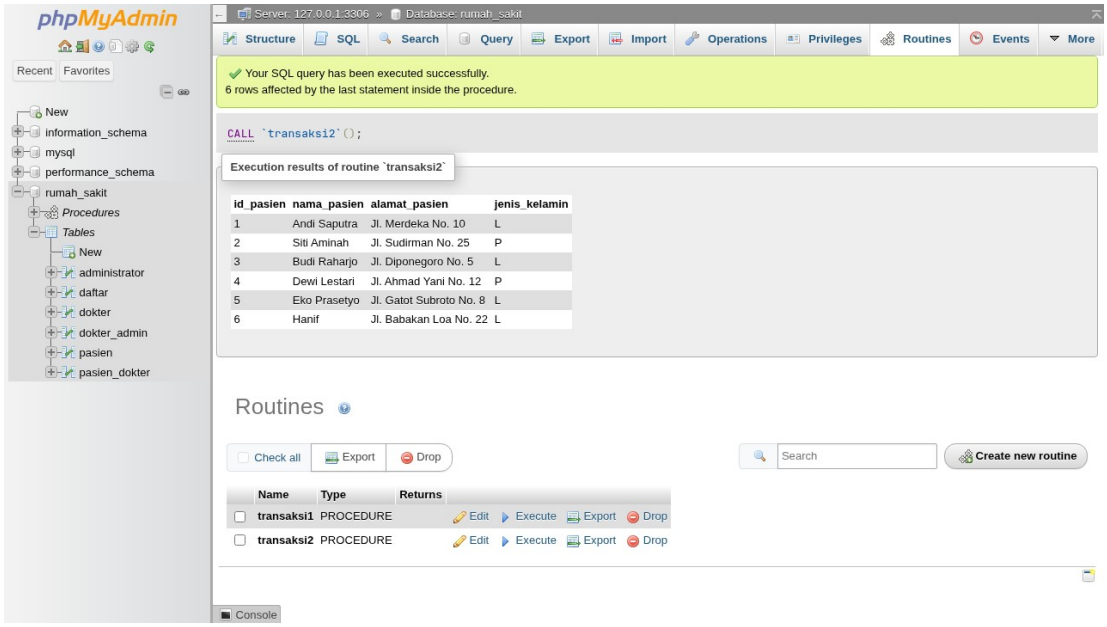


Gambar 5. Pemeriksaan tabel pasien menunjukkan data tetap 5 baris karena transaksi1 belum di-COMMIT.

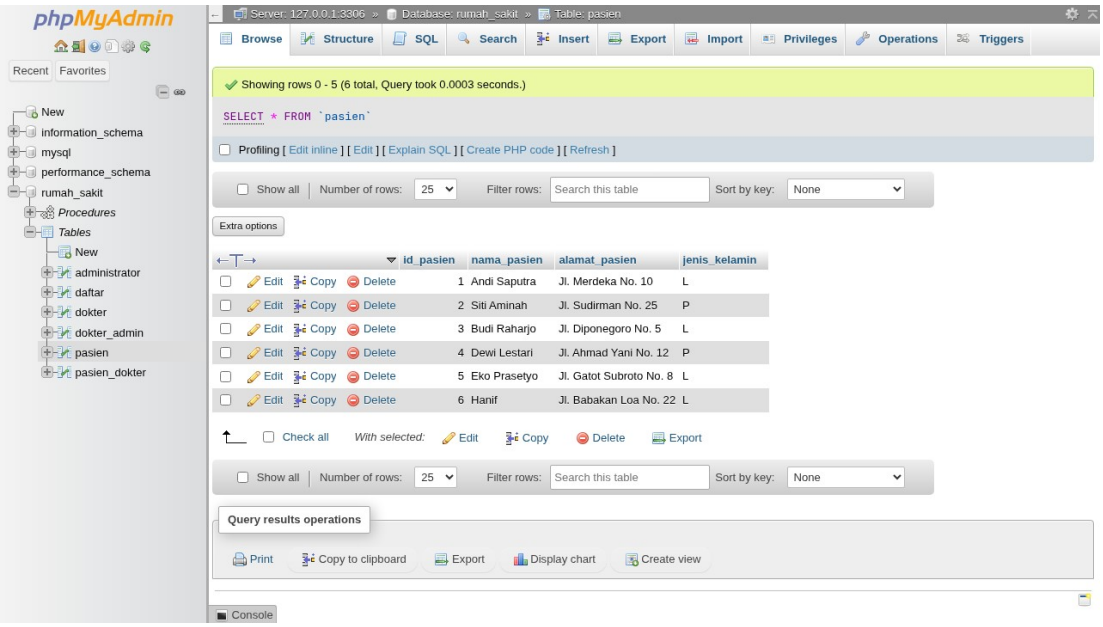
4.3. Pembuatan dan Eksekusi Routine transaksi2 (Dengan Perintah COMMIT): Selanjutnya dibuat routine transaksi2 dengan menambahkan perintah COMMIT setelah INSERT. Setelah dieksekusi, pemeriksaan pada tabel pasien membuktikan bahwa data ID 6 atas nama Hanif telah tersimpan secara permanen.



Gambar 6. Pembuatan Stored Procedure transaksi2 dengan penambahan perintah COMMIT.

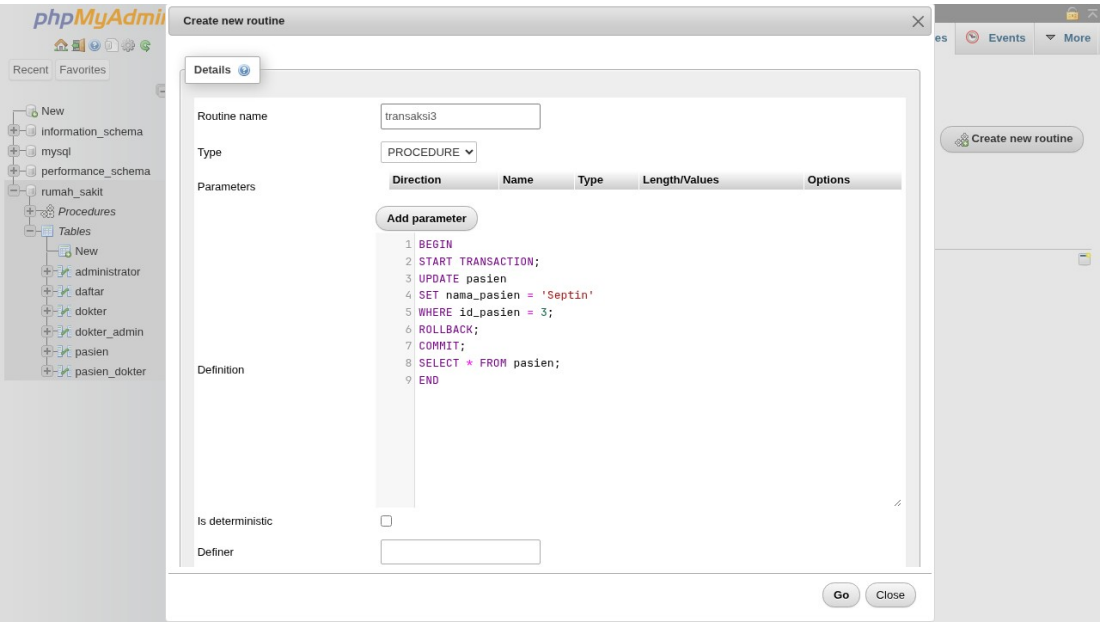


Gambar 7. Hasil eksekusi routine transaksi2.

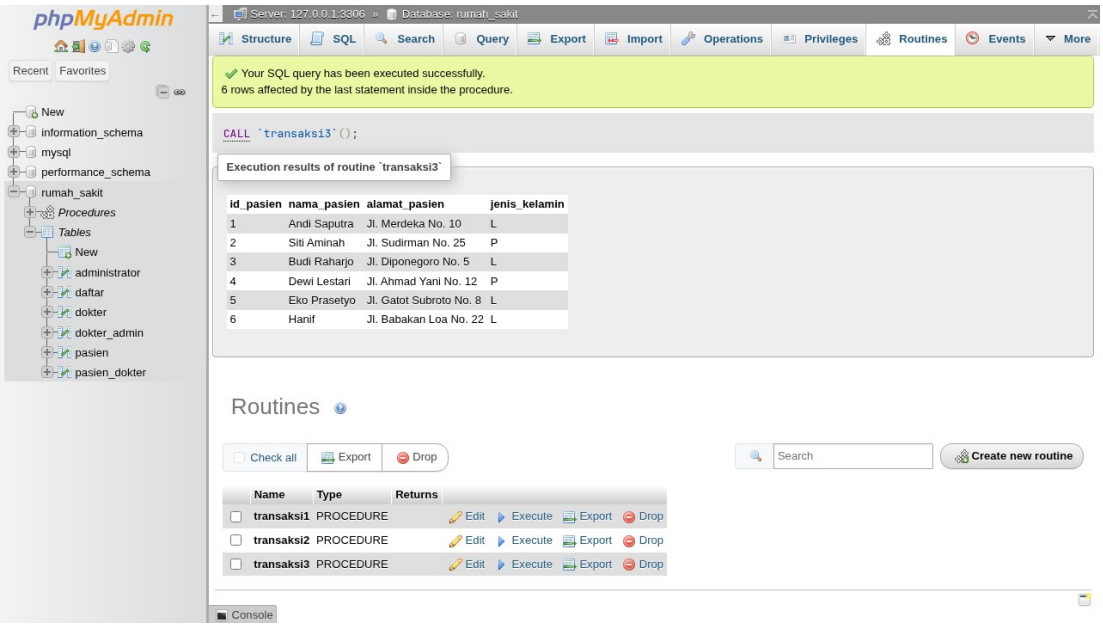


Gambar 8. Tabel pasien setelah transaksi2: data ID 6 (Hanif) berhasil tersimpan permanen.

4.4. Pembuatan dan Eksekusi Routine transaksi3 (Perintah ROLLBACK): Pada transaksi3 dilakukan perubahan (UPDATE) nama pasien ID 3 dari Budi Raharjo menjadi Septin, namun diikuti perintah ROLLBACK. Hasil eksekusi menunjukkan perubahan nama dibatalkan sehingga ID 3 tetap bernama Budi Raharjo.

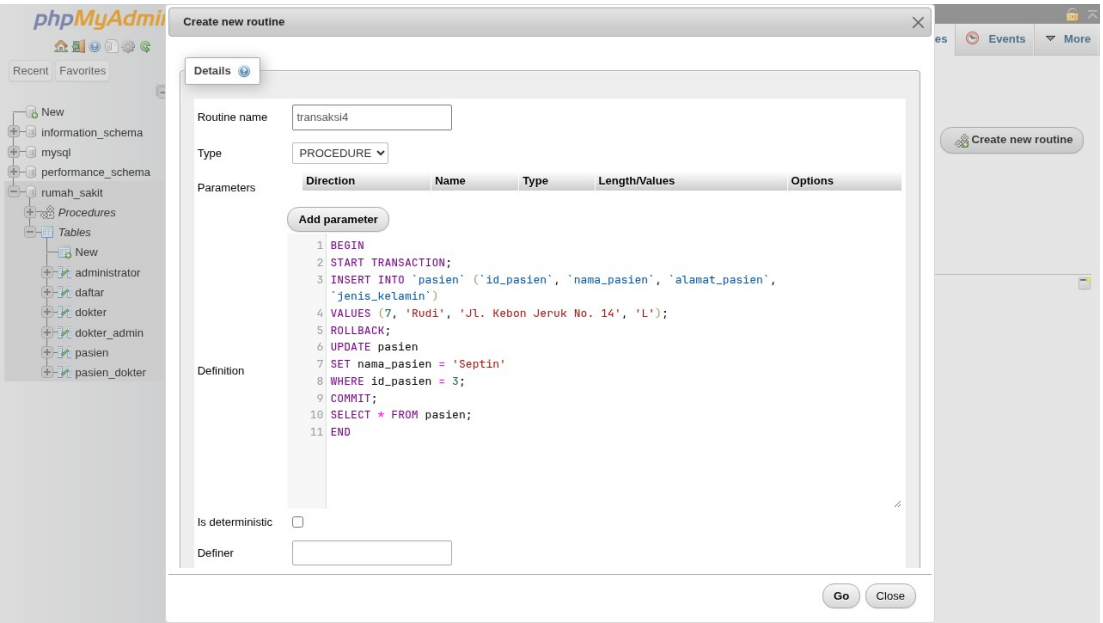


Gambar 9. Pembuatan Stored Procedure transaksi3 menggunakan perintah ROLLBACK.

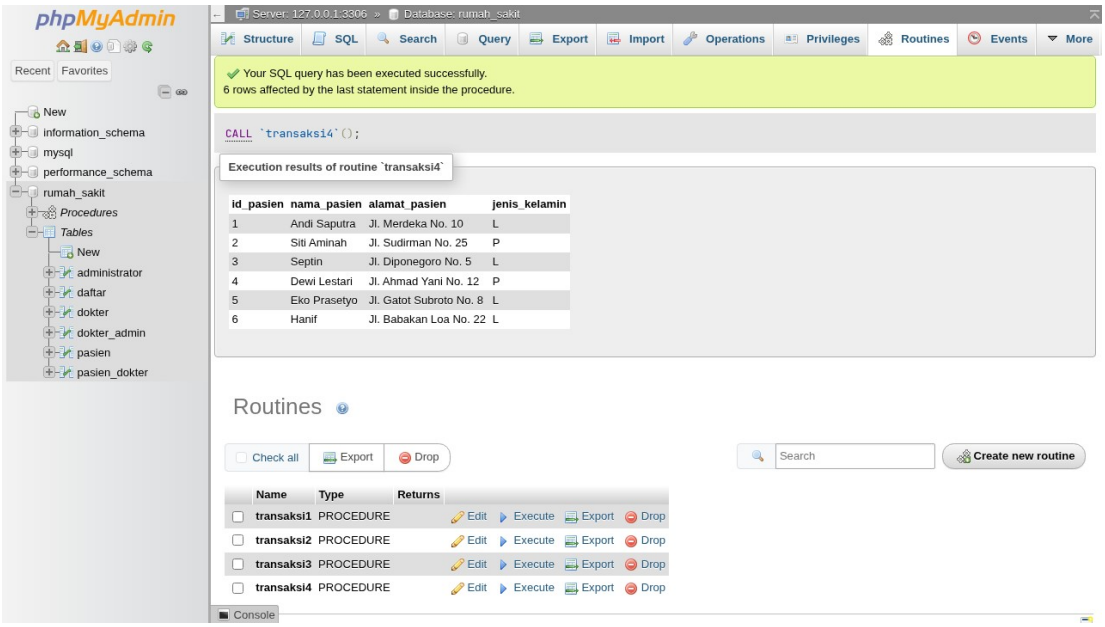


Gambar 10. Hasil eksekusi transaksi3 membuktikan UPDATE dibatalkan oleh ROLLBACK (ID 3 tetap Budi Raharjo).

4.5. Pembuatan dan Eksekusi Routine transaksi4 (Modifikasi ROLLBACK & COMMIT): Sesuai skenario modifikasi ROLLBACK pada modul praktikum, dilakukan INSERT pasien baru ID 7 (Rudi) yang dibatalkan dengan ROLLBACK, lalu dilanjutkan UPDATE nama pasien ID 3 menjadi Septin yang disahkan dengan COMMIT. Hasilnya, pasien ID 7 tidak masuk ke tabel, sedangkan nama pasien ID 3 resmi berubah menjadi Septin.

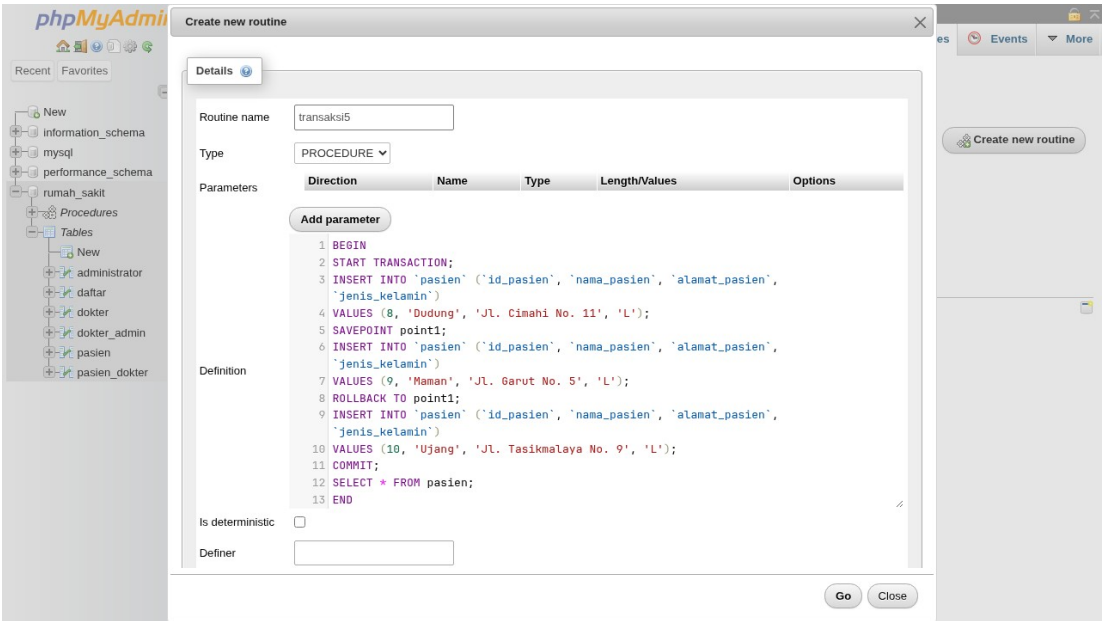


Gambar 11. Pembuatan Stored Procedure transaksi4 (INSERT + ROLLBACK dilanjutkan UPDATE + COMMIT).

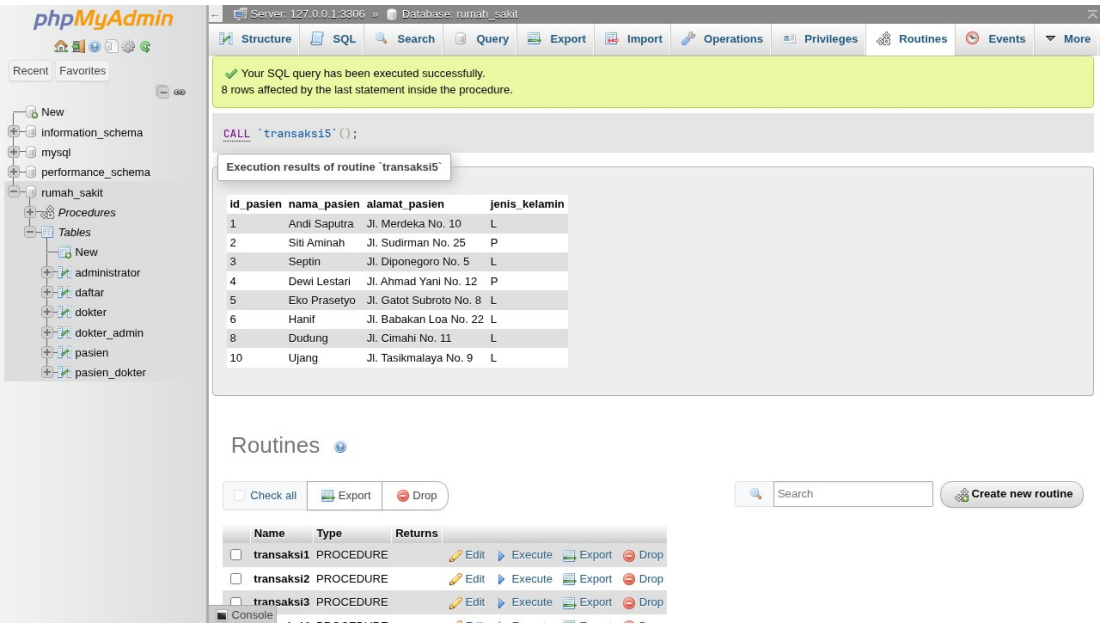


Gambar 12. Hasil eksekusi transaksi4: ID 7 batal masuk karena ROLLBACK, sedangkan ID 3 berubah menjadi Septin karena COMMIT.

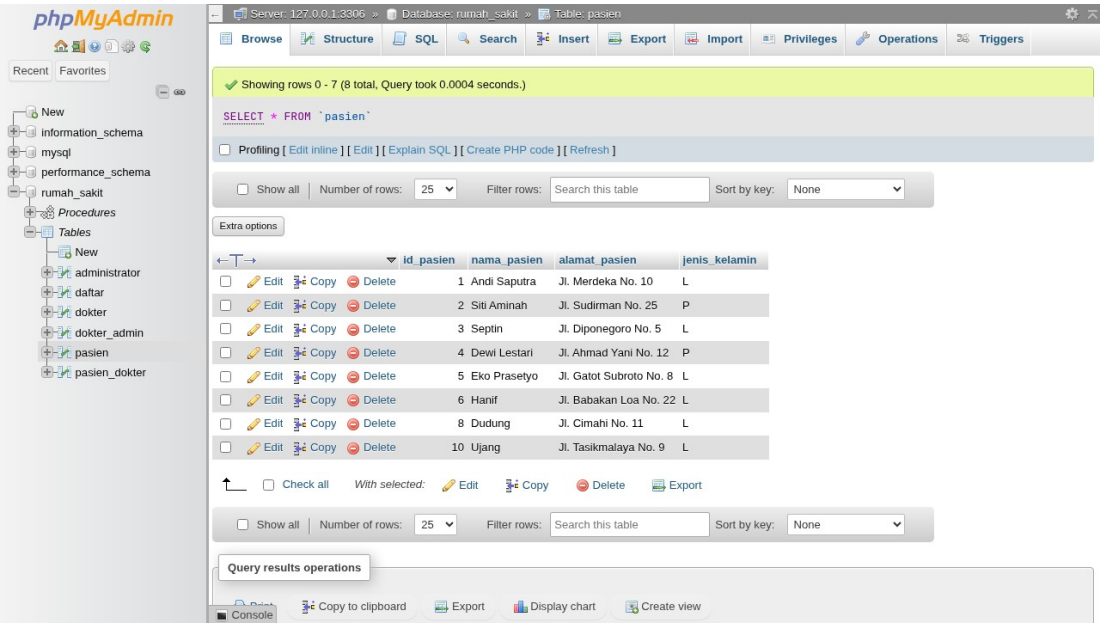
4.6. Pembuatan dan Eksekusi Routine transaksi5 (SAVEPOINT point1 & ROLLBACK TO point1): Pada transaksi5, dilakukan INSERT pasien ID 8 (Dudung) diikuti pembuatan SAVEPOINT point1. Selanjutnya dilakukan INSERT pasien ID 9 (Maman), namun segera dibatalkan kembali ke point1 menggunakan ROLLBACK TO point1, lalu ditambahkan pasien ID 10 (Ujang) dan diakhiri COMMIT. Pemeriksaan tabel pasien membuktikan bahwa ID 8 (Dudung) dan ID 10 (Ujang) tersimpan permanen, sedangkan ID 9 (Maman) dibatalkan.



Gambar 13. Pembuatan Stored Procedure transaksi5 dengan SAVEPOINT point1 dan ROLLBACK TO point1.

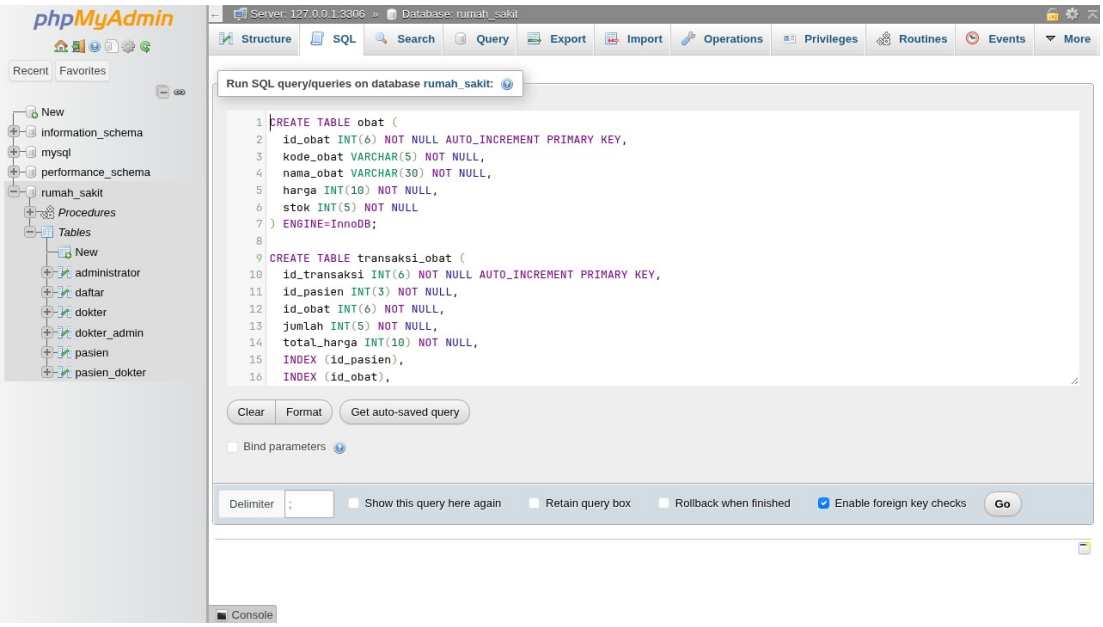


Gambar 14. Hasil eksekusi routine transaksi5.

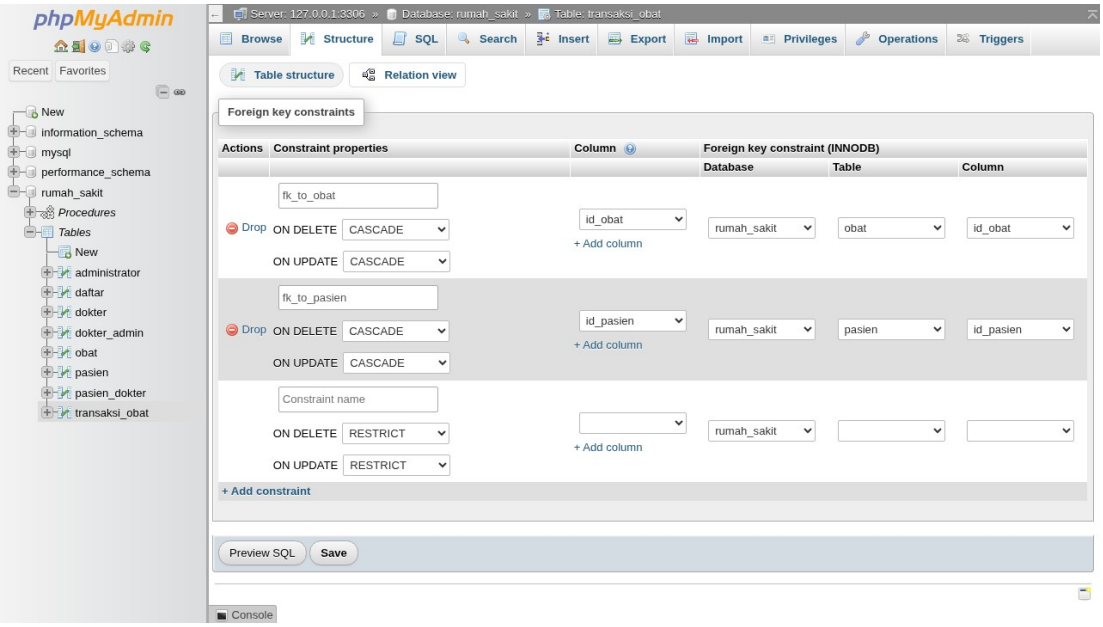


Gambar 15. Tabel pasien setelah transaksi5: ID 8 (Dudung) dan ID 10 (Ujang) tersimpan, sedangkan ID 9 (Maman) batal masuk.

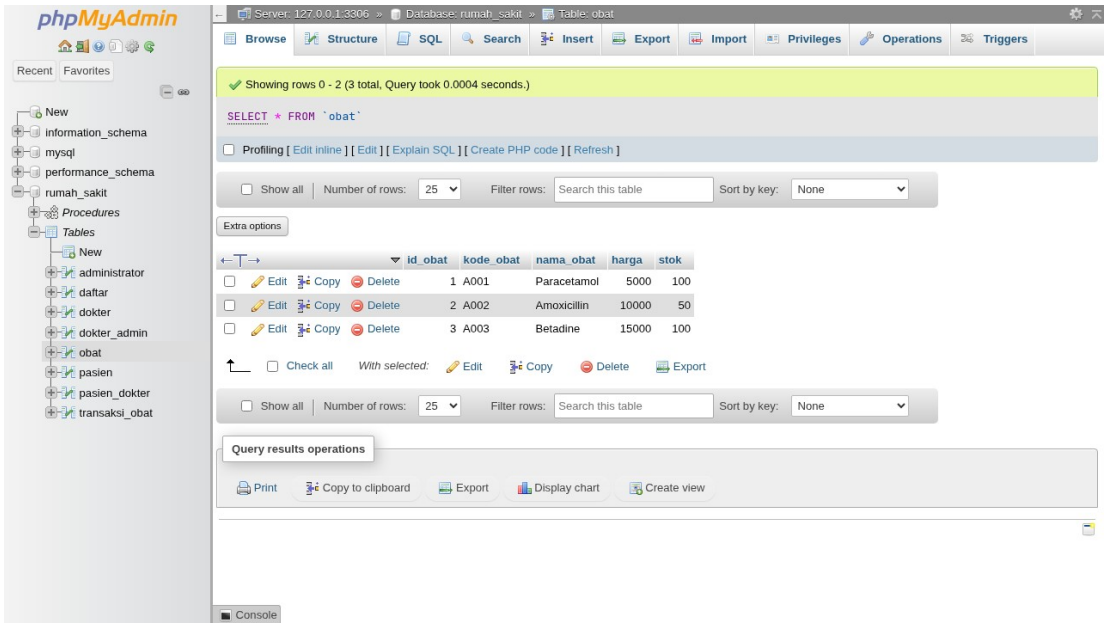
4.7. Pembuatan Tabel obat & transaksi_obat, Relasi Foreign Key, serta Trigger tg_stok_obat: Setelah kelima skenario transaksi selesai, dibuat tabel obat dan transaksi_obat, diatur relasi Foreign Key (ON DELETE CASCADE, ON UPDATE CASCADE), diisi 3 data obat awal (Paracetamol stok 100, Amoxicillin stok 50, Betadine stok 100), serta dibuat Trigger tg_stok_obat (AFTER INSERT). Saat diuji dengan menginput pembelian 15 unit Paracetamol oleh pasien ID 6 (Hanif), stok Paracetamol otomatis berkurang dari 100 menjadi 85.



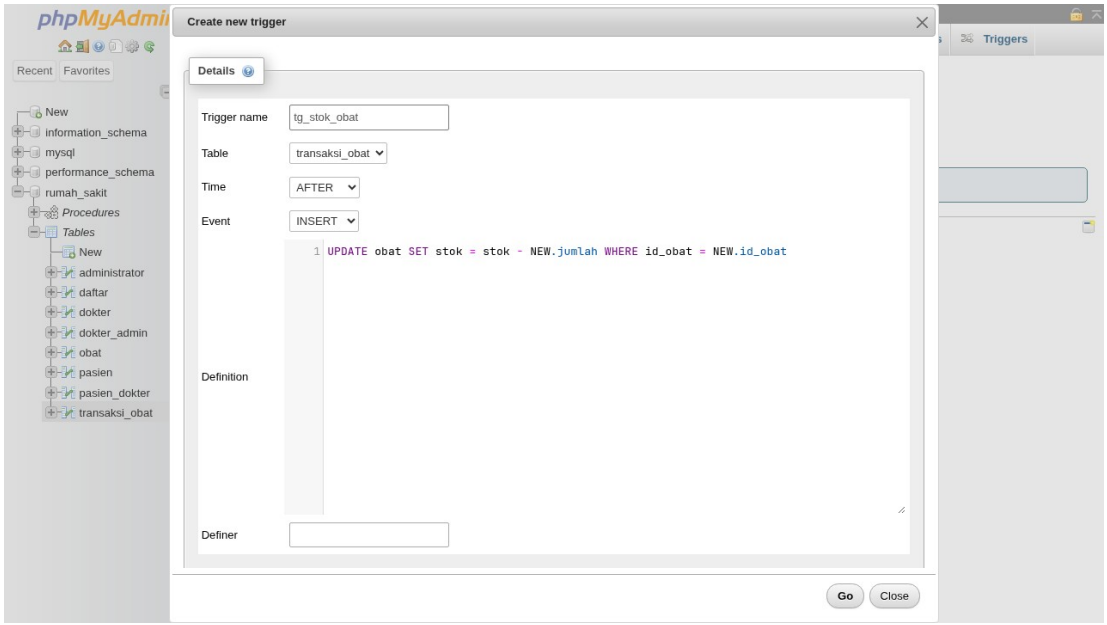
Gambar 16. Pembuatan tabel obat dan tabel transaksi_obat melalui tab SQL.



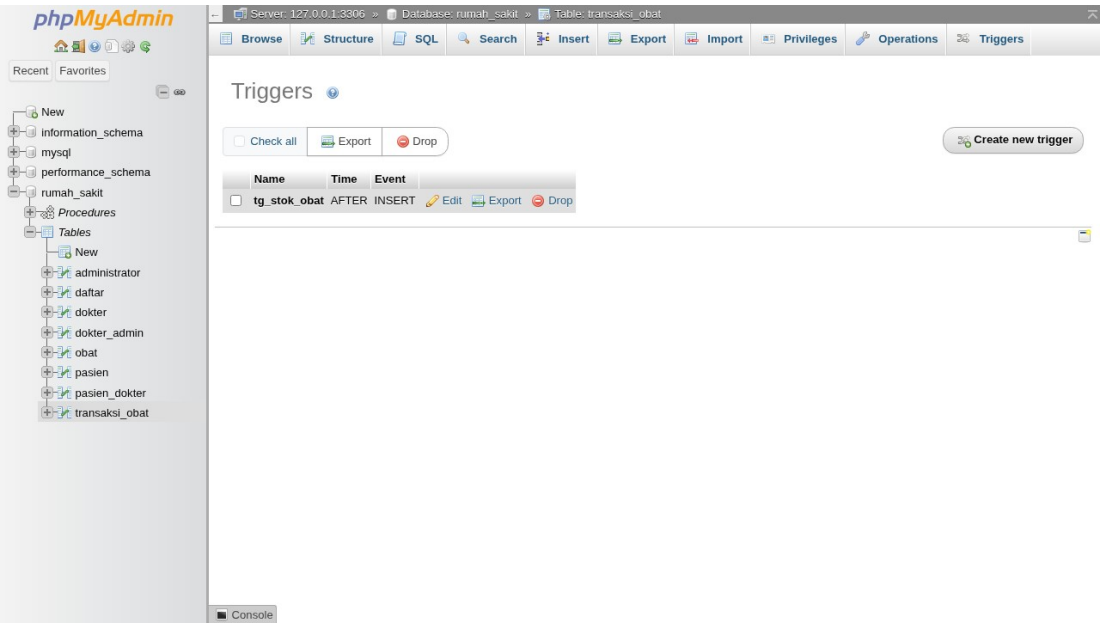
Gambar 17. Pengaturan relasi Foreign Key (CASCADE) pada tabel transaksi_obat ke tabel pasien dan obat.



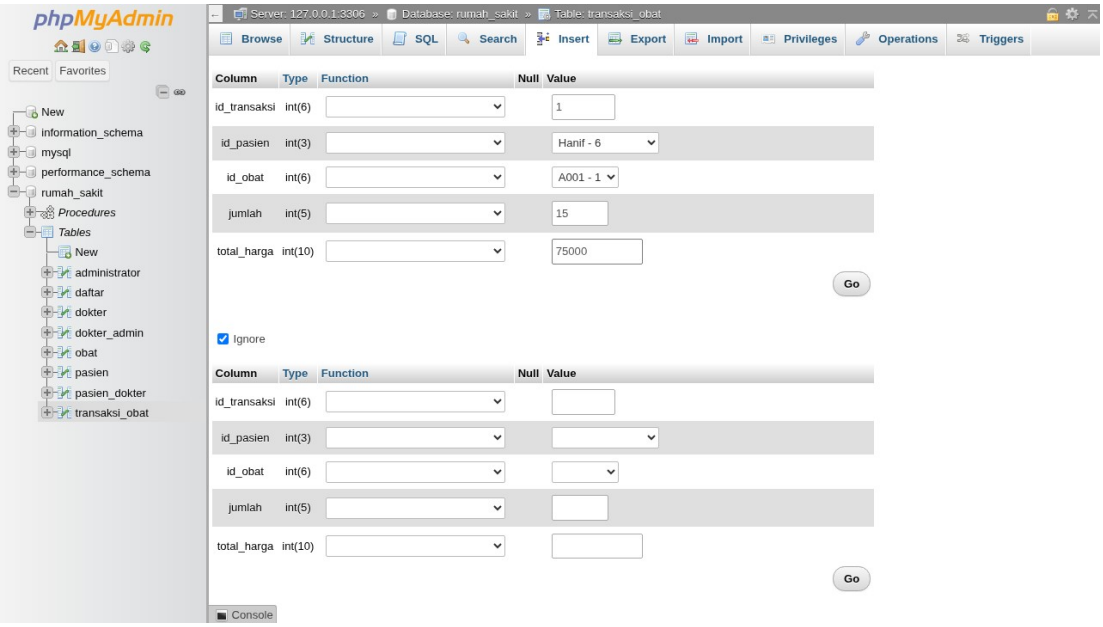
Gambar 18. Data stok awal pada tabel obat (Paracetamol = 100, Amoxicillin = 50, Betadine = 100).



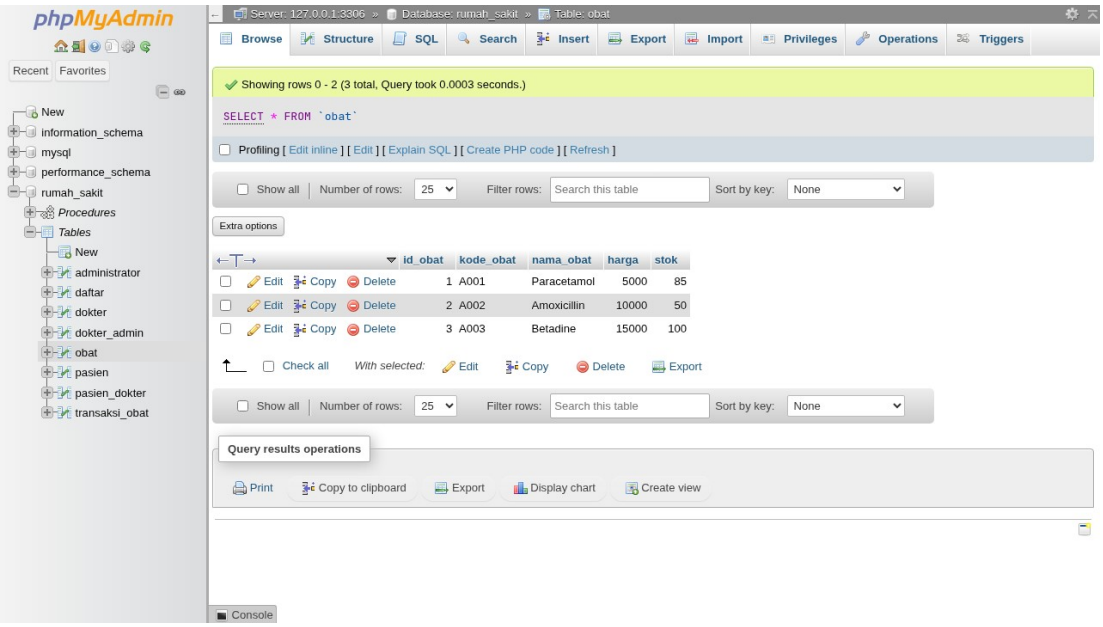
Gambar 19. Pembuatan Trigger tg_stok_obat (AFTER INSERT) pada tabel transaksi_obat.



Gambar 20. Trigger tg_stok_obat berhasil terdaftar aktif pada tabel transaksi_obat.



Gambar 21. Pengisian form Insert pada tabel transaksi_obat (pembelian 15 unit Paracetamol oleh pasien ID 6).



Gambar 22. Bukti stok Paracetamol pada tabel obat otomatis berkurang dari 100 menjadi 85.

5. Dokumentasi Langkah demi Langkah Praktikum B: Basis Data Terdistribusi (Replikasi Master-Slave)

Pada Praktikum Basis Data Terdistribusi, dilakukan konfigurasi replikasi satu arah dari Server Master pada Host Windows (IP 192.168.56.1) ke Server Slave pada Guest Linux Ubuntu di VirtualBox (IP 192.168.56.101) dengan konfigurasi dan perintah sebagai berikut:

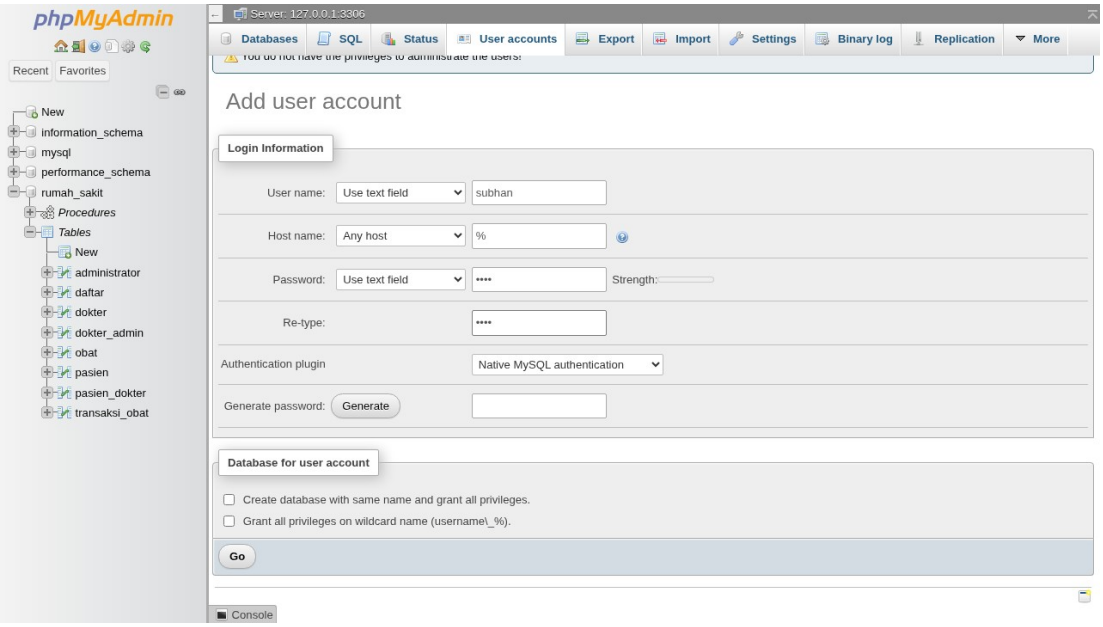
```
# 1. Konfigurasi my.ini pada Server Master Windows (IP: 192.168.56.1)
[mysqld]
log-bin = mysql-bin
server-id = 1
binlog-do-db = rumah_sakit

# 2. Konfigurasi /opt/lampp/etc/my.cnf pada Server Slave Ubuntu (IP:
192.168.56.101)
[mysqld]
server-id = 2
replicate-do-db = rumah_sakit

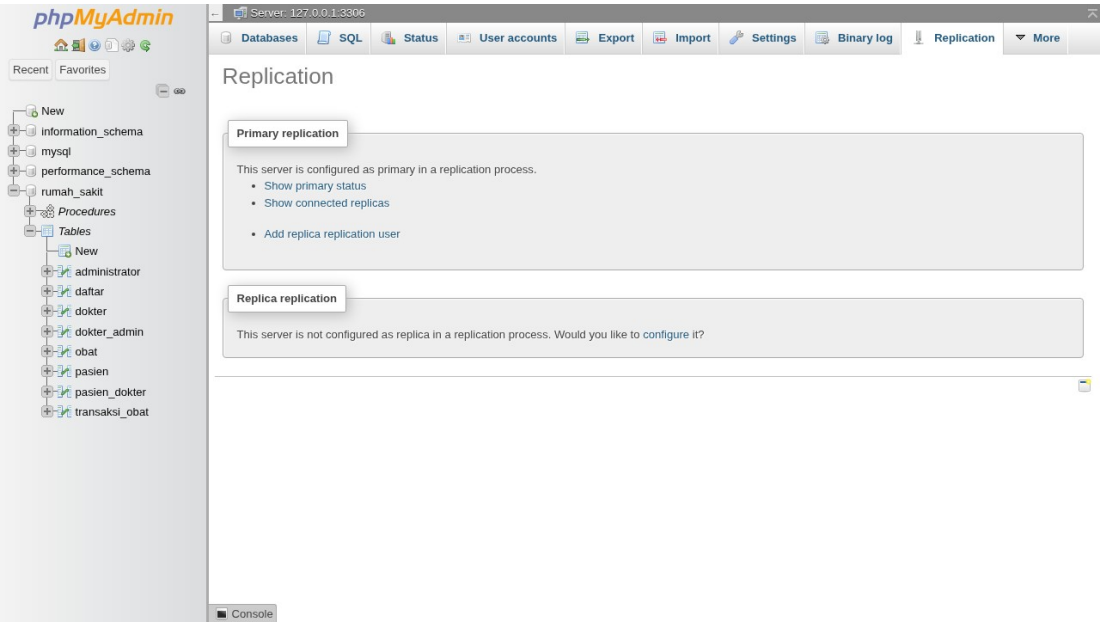
-- 3. Query Koneksi Slave Ubuntu ke Master Windows
STOP SLAVE;
CHANGE MASTER TO
  MASTER_HOST='192.168.56.1',
  MASTER_USER='subhan',
  MASTER_PASSWORD='1234',
  MASTER_LOG_FILE='mysql-bin.000001',
  MASTER_LOG_POS=328;
START SLAVE;
SHOW SLAVE STATUS\G
```

5.1. Pembuatan User Replikasi, Konfigurasi Primary Replication, dan Ekspor Database di Master: Pada phpMyAdmin Master dibuat user subhan dengan host % (Any host) serta diberikan hak akses

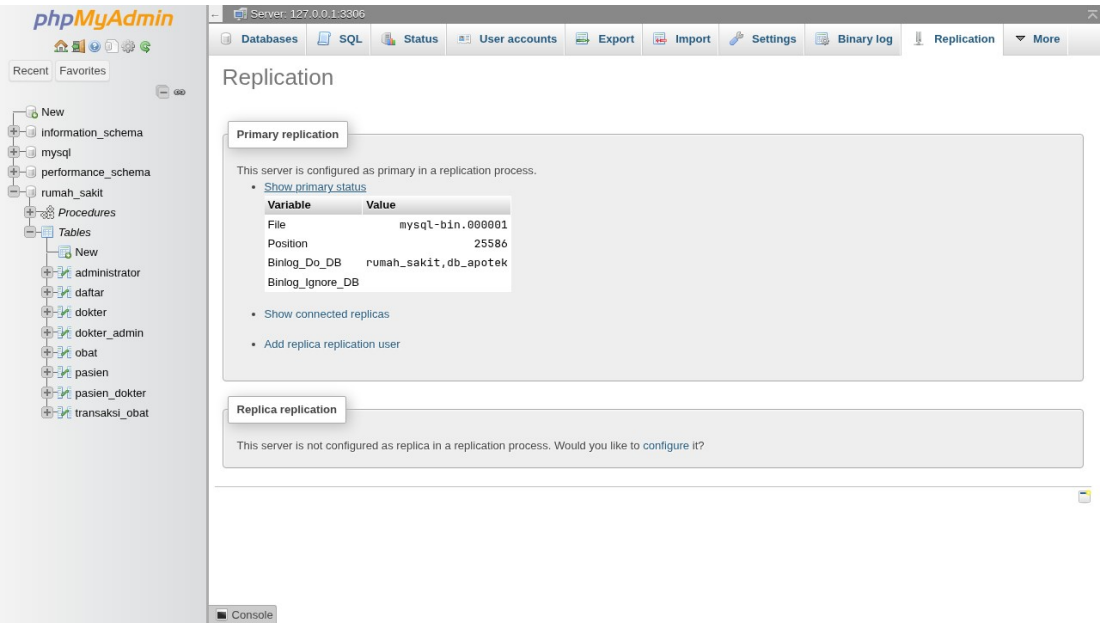
Replication client dan Replication slave. Setelah konfigurasi my.ini dipasang dan MySQL direstart, status Primary replication aktif pada file mysql-bin.000001. Selanjutnya database rumah_sakit diekspor menjadi file rumah_sakit.sql.



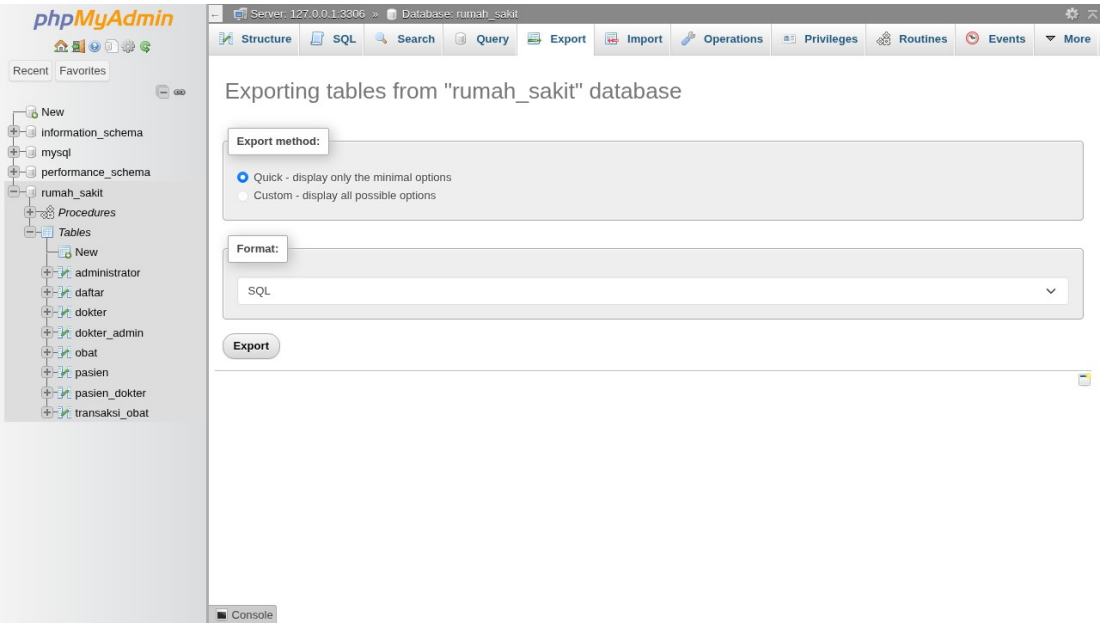
Gambar 23. Pembuatan akun user replikasi (subhan) dengan hak akses Replication client dan Replication slave.



Gambar 24. Status menu Replication pada Server Master setelah konfigurasi my.ini (server-id = 1).

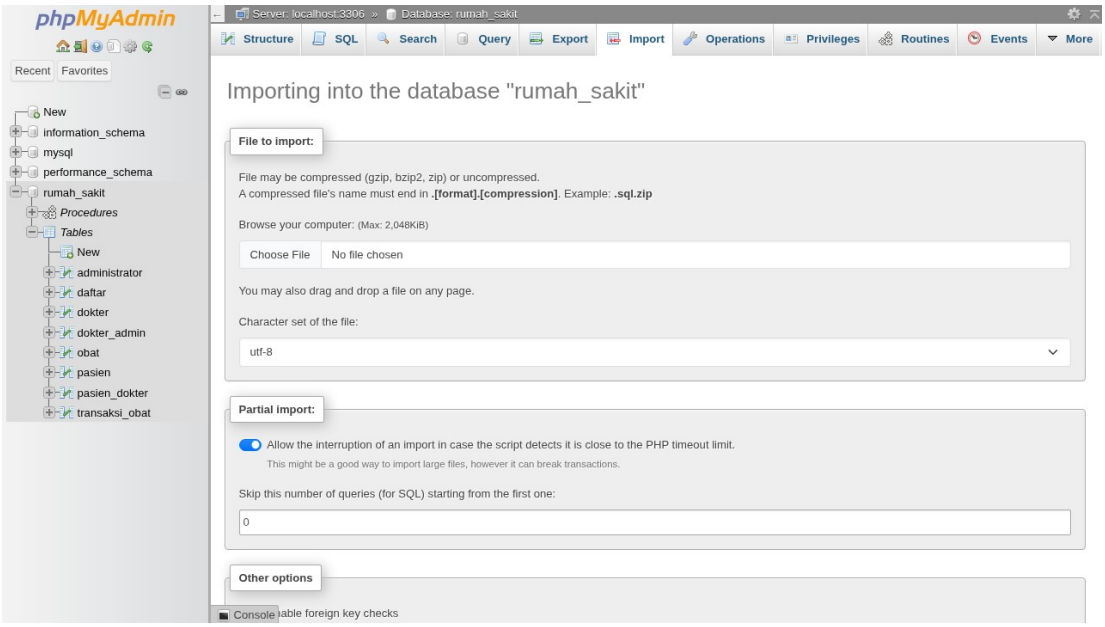


Gambar 25. Tabel Show primary status pada Server Master menampilkan File mysql-bin.000001 beserta Position.

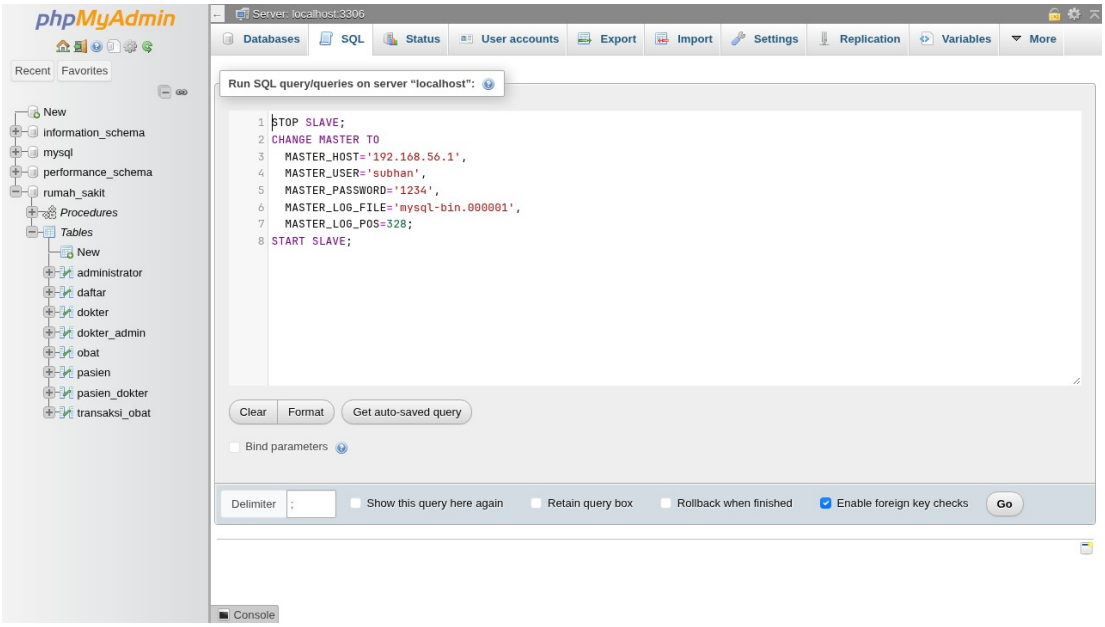


Gambar 26. Proses Export database rumah_sakit dari Server Master untuk dipindahkan ke Ubuntu.

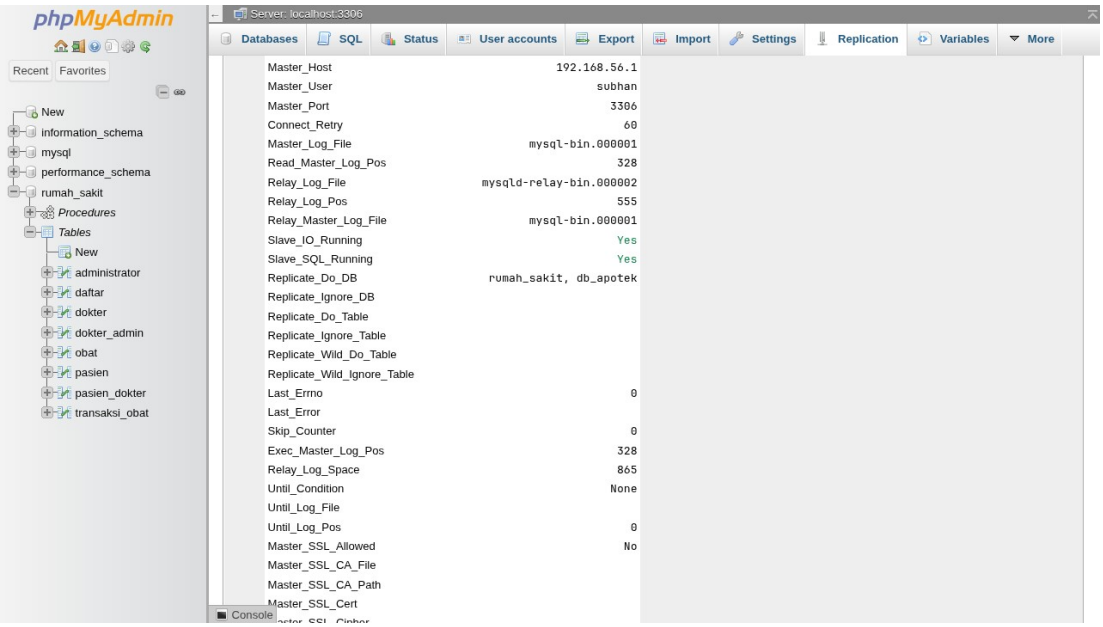
5.2. Impor Database, Eksekusi CHANGE MASTER TO, dan Verifikasi Status Slave di Ubuntu: Pada phpMyAdmin Slave di Ubuntu, database rumah_sakit dibuat dan file rumah_sakit.sql diimpor. Setelah konfigurasi my.cnf (server-id = 2) aktif, dijalankan perintah STOP SLAVE, CHANGE MASTER TO, dan START SLAVE. Tabel status replica pada Slave Ubuntu menampilkan Slave_IO_Running: Yes dan Slave_SQL_Running: Yes.



Gambar 27. Proses Import database rumah_sakit.sql pada phpMyAdmin Server Slave Ubuntu.

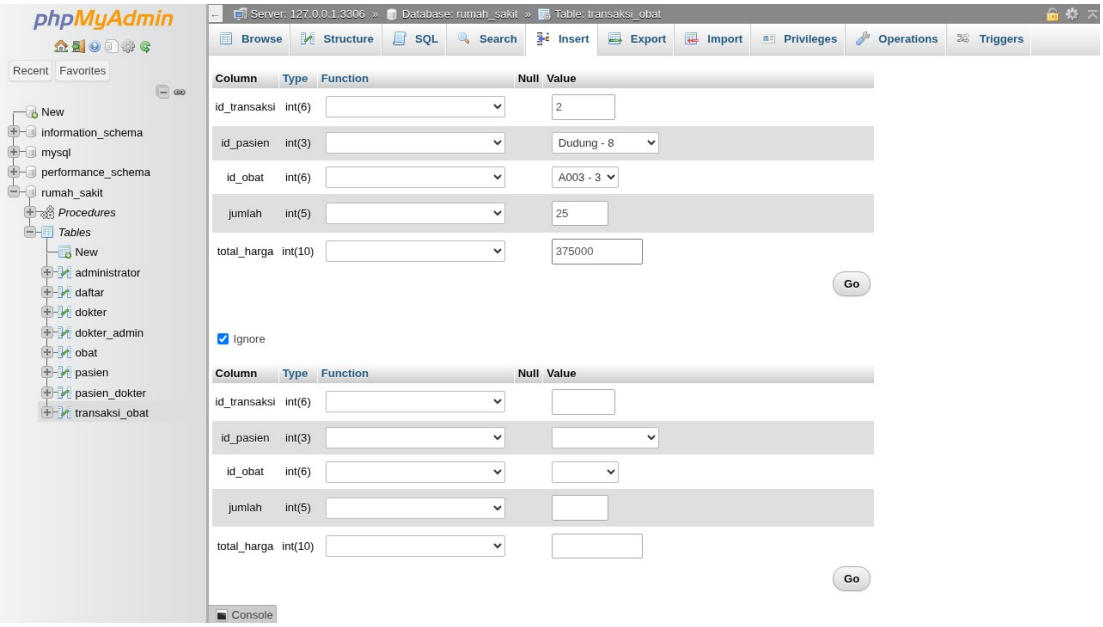


Gambar 28. Eksekusi perintah CHANGE MASTER TO dan START SLAVE pada tab SQL Server Slave Ubuntu.

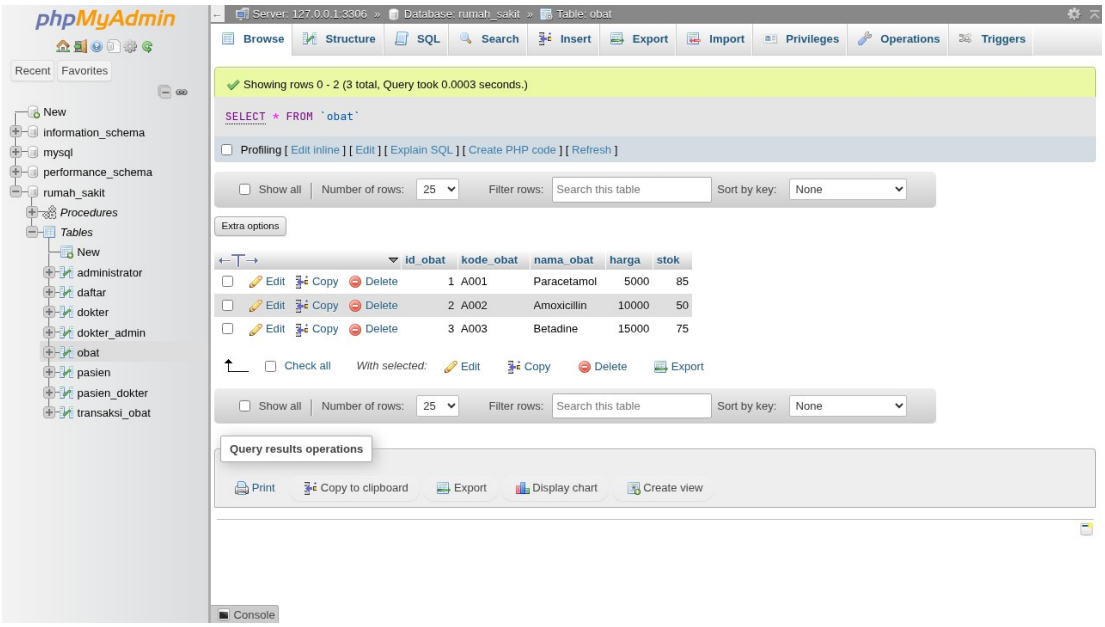


Gambar 29. Bukti status replikasi pada Server Slave Ubuntu: Master_Host = 192.168.56.1, Slave_IO_Running = Yes, dan Slave_SQL_Running = Yes.

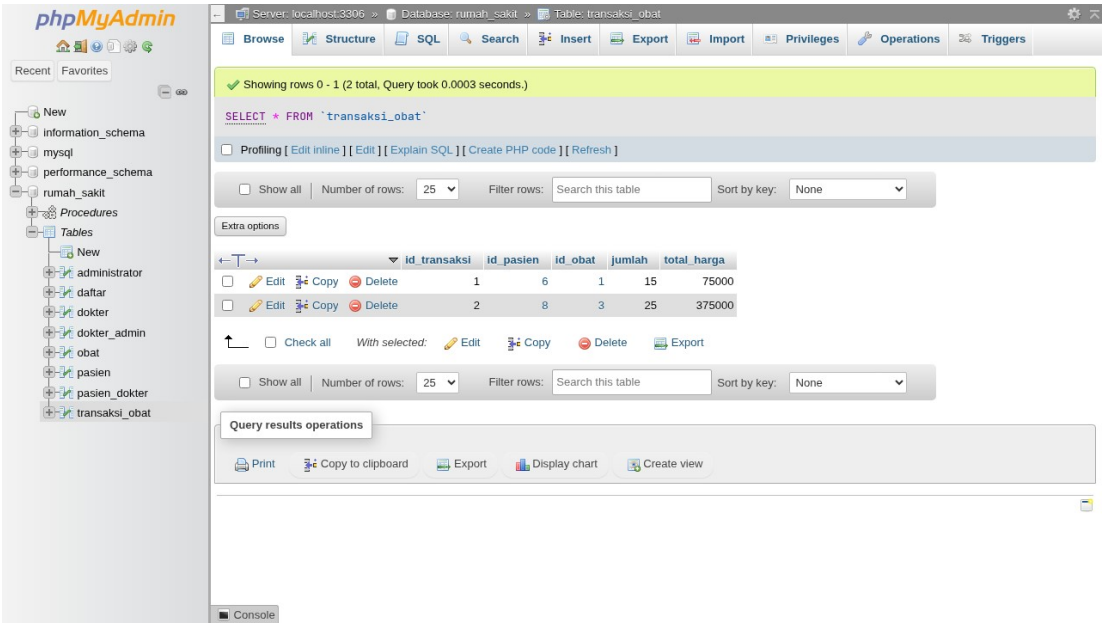
5.3. Pengujian Sinkronisasi Data Real-Time dari Master ke Slave: Untuk menguji replikasi, dilakukan input transaksi ke-2 pada tabel transaksi_obat di Server Master (pembelian 25 unit Betadine oleh pasien ID 8 atas nama Dudung). Stok Betadine di Master turun dari 100 menjadi 75. Saat tabel transaksi_obat dan tabel obat pada Server Slave Ubuntu di-refresh, data transaksi ke-2 otomatis masuk dan stok Betadine di Slave Ubuntu otomatis ikut berkurang menjadi 75.



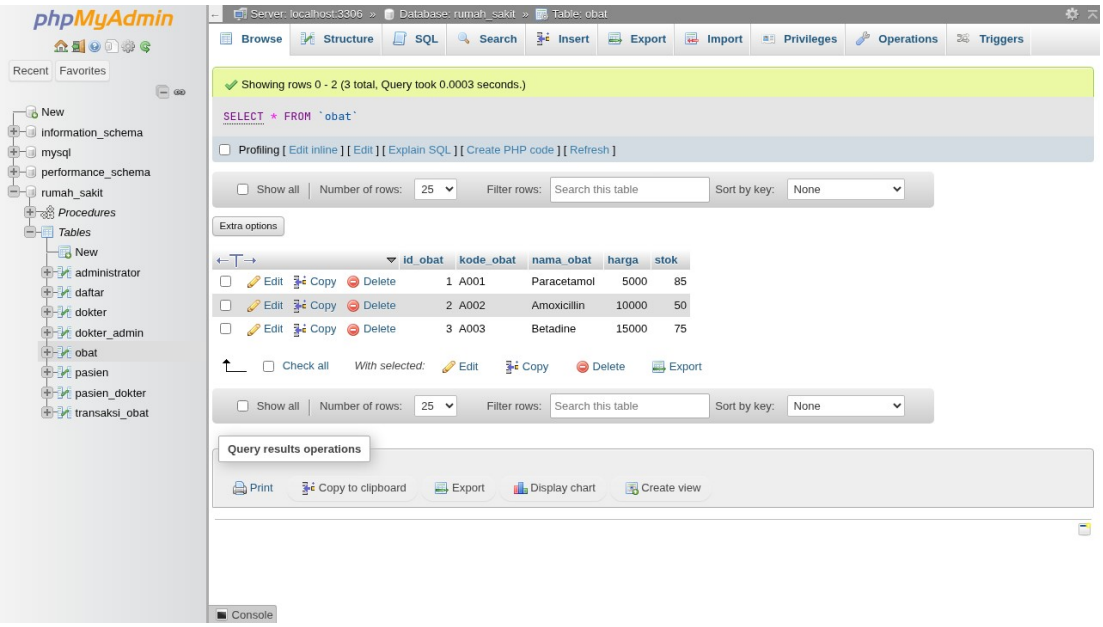
Gambar 30. Pengisian form Insert transaksi ke-2 (pembelian 25 unit Betadine oleh pasien ID 8) pada Server Master.



Gambar 31. Stok Betadine pada tabel obat di Server Master berkurang dari 100 menjadi 75.



Gambar 32. Data transaksi ke-2 otomatis muncul pada tabel transaksi_obat di Server Slave Ubuntu.



Gambar 33. Stok Betadine pada tabel obat di Server Slave Ubuntu otomatis ikut berubah menjadi 75.

6. Kesimpulan

Berdasarkan seluruh rangkaian praktikum Tugas 3 Mata Kuliah Basis Data yang telah dilaksanakan secara bertahap, dapat disimpulkan bahwa pengelolaan transaksi pada MySQL/MariaDB menggunakan storage engine InnoDB berhasil menjamin keutuhan data (ACID). Perintah START TRANSACTION yang diikuti COMMIT menyimpan perubahan data secara permanen (transaksi2), sedangkan transaksi tanpa COMMIT (transaksi1) maupun transaksi yang diikuti ROLLBACK (transaksi3 dan transaksi4) membatalkan mutasi yang belum disahkan. Penggunaan SAVEPOINT dan ROLLBACK TO SAVEPOINT (transaksi5) memungkinkan pembatalan parsial ke titik aman tertentu tanpa menggugurkan keseluruhan transaksi, sementara Trigger AFTER INSERT (tg_stok_obat) secara otomatis memotong stok pada tabel obat setiap kali terjadi transaksi pembelian baru. Pada praktikum Basis Data Terdistribusi, konfigurasi Replikasi Master-Slave antara Host Windows (192.168.56.1, server-id = 1) dan Guest Linux Ubuntu di VirtualBox (192.168.56.101, server-id = 2) terbukti berhasil berjalan secara real-time dengan indikator Slave_IO_Running: Yes dan Slave_SQL_Running: Yes.

Sumber Referensi:

Nurhayati, O. D., & Tim Penyusun. (2022). MSIM4206 – Basis Data (Edisi 2). Tangerang Selatan: Universitas Terbuka.

Tim Pengembang e-Learning UT. (2025). Panduan Praktikum Mata Kuliah Berpraktik MSIM4206 Basis Data: Praktikum Transaksi dan Basis Data Terdistribusi. Tangerang Selatan: Universitas Terbuka.

MariaDB Foundation. (2024). Standard Replication and Binary Log Documentation for MariaDB 10.4. Diakses dari https://mariadb.com/kb/en/standard-replication/